

```
In [ ]: import pandas as pd
```

01 CFPB Stratified Sampling and Feature Preparation

Objective: construct a reproducible, proportionally stratified CFPB sample that preserves Product x Year-Quarter x Region structure for downstream EDA and forecasting notebooks.

This notebook prepares the following artifacts:

- engineered temporal and geographic features (`year_quarter` , `region` , `stratum`)
- exploratory state-level and final region-level stratification diagnostics
- final 300k stratified sample exported for exploratory data analysis (notebook 02)

Step 1: Creating Year-Quarter

```
In [ ]: df['Date received'] = pd.to_datetime(df['Date received'], errors='coerce')

df['year_quarter'] = (
    df['Date received']
        .dt.to_period('Q')
        .astype(str)
)
```

This gives values like '2021Q1', '2022Q3', etc.

Step 2: Define Initial Geography Field

Before moving to regional aggregation, a state-level geography field is created to support exploratory strata design and sparsity checks.

```
In [ ]: df['geo'] = df['State']
```

Step 3: Mapping State to Region

For the final workflow, I use U.S. Census Bureau regions instead of individual states. This reduces sparsity while still preserving meaningful geographic structure for the downstream sample.

1. Northeast

- CT, ME, MA, NH, RI, VT
- NJ, NY, PA

2. Midwest

- IL, IN, MI, OH, WI
- IA, KS, MN, MO, NE, ND, SD

3. South

- DE, FL, GA, MD, NC, SC, VA, DC, WV
- AL, KY, MS, TN
- AR, LA, OK, TX

4. West

- AZ, CO, ID, MT, NV, NM, UT, WY
- AK, CA, HI, OR, WA

5. Other

- Any remaining values in State (e.g., territories or non-state codes)

```
In [ ]: northeast = ['CT', 'ME', 'MA', 'NH', 'RI', 'VT', 'NJ', 'NY', 'PA']
midwest   = ['IL', 'IN', 'MI', 'OH', 'WI', 'IA', 'KS', 'MN', 'MO', 'NE', 'ND', 'SD']
south     = ['DE', 'FL', 'GA', 'MD', 'NC', 'SC', 'VA', 'DC', 'WV',
            'AL', 'KY', 'MS', 'TN', 'AR', 'LA', 'OK', 'TX']
west      = ['AZ', 'CO', 'ID', 'MT', 'NV', 'NM', 'UT', 'WY', 'AK', 'CA', 'HI', 'OR', 'WA']

def map_region(state):
    if state in northeast:
        return 'Northeast'
    elif state in midwest:
        return 'Midwest'
    elif state in south:
        return 'South'
    elif state in west:
        return 'West'
    else:
        return 'Other'

df['region'] = df['State'].apply(map_region)
```

Rows with unresolved geography are removed so each record can be assigned to a valid regional stratum in later steps.

```
In [ ]: # Drop rows without valid region
df = df.dropna(subset=['region'])
```

Regional share is reviewed here to confirm that the aggregated geography feature is coherent and suitable for proportional sampling.

```
In [ ]: df['region'].value_counts(normalize=True)
```

Step 4: Finalize Geography for Production Strata

The region field is adopted as the production geography feature to balance interpretability and stratum stability.

```
In [ ]: df['year_quarter'] = pd.to_datetime(df['Date received']).dt.to_period('Q').astype(s
df['stratum'] = (
    df['Product'].astype(str) + "|" +
    df['year_quarter'].astype(str) + "|" +
    df['region'].astype(str)
)
```

Step 5: Sanity Check Stratum Granularity

A quick cardinality check ensures strata are neither too coarse (loss of structure) nor too sparse (unstable sampling allocations).

```
In [ ]: num_unique_strata = df['stratum'].nunique()
print(f"Total number of unique strata: {num_unique_strata:,}")
```

```
In [ ]: strata_counts = df.groupby(['Product', 'year_quarter', 'region']).size()

strata_counts.describe()
```

Interpretation:

- The median stratum size is sufficiently large for stable proportional sampling.
- The distribution remains right-skewed, which is expected in CFPB data (heavy concentration in specific product-time-region combinations).
- A small long-tail of low-count strata is acceptable when the share of sparse strata is minimal.

```
In [ ]: print(f"Number of strata with fewer than 20 samples: {(strata_counts < 20).sum():,}")
```

Conclusion from diagnostics: the current stratum design is appropriate for sampling, with manageable sparsity and strong representational coverage.

2. Stratified Sampling Workflow

With core time and geography features prepared, the notebook now defines strata, allocates proportional sample sizes, and validates that sampled distributions closely match the source population.

Step 1: Dropping Missing Key Fields

For the initial state-level sampling pass, stratified sampling only works cleanly if strata are defined, so I dropped rows that had any of the specified strata info missing.

```
In [ ]: df_strata = df.dropna(subset=['Product', 'year_quarter', 'geo'])
```

Step 2: Creating a Strata Key

Next, I ensured that each unique Product x Year-Quarter x State combination is represented as one stratum, so that the exploratory stratified sampling works cleanly.

```
In [ ]: df_strata['stratum'] = (
    df_strata['Product'].astype(str) + '|' +
    df_strata['year_quarter'].astype(str) + '|' +
    df_strata['geo'].astype(str)
)
```

Step 3: Computing Stratum Sizes

```
In [ ]: stratum_counts = df_strata['stratum'].value_counts()

total_rows = len(df_strata)
```

Step 4: Proportional Stratified Sampling

For a target size (say 500,000): $n_{\text{stratum}} = (N_{\text{stratum}} / N_{\text{total}}) \times N_{\text{target}}$

```
In [ ]: import numpy as np

def stratified_sample(df, target_size, random_state=42):
    df = df.copy()

    # Calculate proportions
    stratum_counts = df['stratum'].value_counts()
    total = len(df)

    # Target sample per stratum
    stratum_sample_sizes = (
        (stratum_counts / total) * target_size
    ).round().astype(int)

    # Merge sizes back
    sample_map = stratum_sample_sizes.to_dict()
    df['sample_size'] = df['stratum'].map(sample_map)

    # Sample within each stratum
```

```

sampled_df = (
    df.groupby('stratum', group_keys=False)
      .apply(lambda x: x.sample(
          n=min(len(x), sample_map[x.name]),
          random_state=random_state
        ))
)

return sampled_df.drop(columns=['sample_size'])

```

Step 5: Generate Candidate Samples

Two sample sizes are generated for comparison:

- 500k: broader coverage for diagnostics and robustness checks
- 300k: computationally efficient size selected for the core EDA workflow

```

In [ ]: # 500k sample generation
df_500k = stratified_sample(df_strata, 500_000)

# 300k sample generation
df_300k = stratified_sample(df_strata, 300_000)

```

Step 6: Validate Stratification Fidelity

This step compares marginal distributions between the source population and sampled data to ensure proportional stratification behavior is preserved.

```

In [ ]: def compare_distribution(original, sample, col):
    orig_dist = original[col].value_counts(normalize=True)
    sample_dist = sample[col].value_counts(normalize=True)
    return pd.concat([orig_dist, sample_dist], axis=1, keys=['original', 'sample'])

```

```

In [ ]: compare_distribution(df_strata, df_500k, 'Product')
compare_distribution(df_strata, df_500k, 'year_quarter')
compare_distribution(df_strata, df_500k, 'geo')

```

3. Final Sample Specification

Final design decisions for downstream notebooks:

1. Stratification keys: `Product`, `year_quarter`, `region`
2. Allocation method: proportional allocation with rounding-drift correction
3. Final output size: 300,000 rows

Step 1: Defining Strata

```
In [ ]: strata_cols = ['Product', 'year_quarter', 'region']
```

Step 2: Computing Proportional Allocation (Adjusting for Rounding Drift)

```
In [ ]: # Target sample size
TARGET_N = 300_000

# Get stratum size
strata_counts = df.groupby(strata_cols).size().reset_index(name='count')

# Compute proportions
total_records = len(df)

strata_counts['prop'] = strata_counts['count'] / total_records
strata_counts['sample_n'] = (strata_counts['prop'] * TARGET_N).round().astype(int)

# Adjust for Rounding Drift
difference = TARGET_N - strata_counts['sample_n'].sum()

if difference != 0:
    strata_counts.loc[strata_counts.sample(difference).index, 'sample_n'] += 1
```

Step 3: Merging Back & Creating Sample

```
In [ ]: # Merge sample sizes into main df
df = df.merge(
    strata_counts[strata_cols + ['sample_n']],
    on=strata_cols,
    how='left'
)
```

```
In [ ]: # Sample within each stratum
df_sample_300k = (
    df.groupby(strata_cols, group_keys=False)
      .apply(lambda x: x.sample(
          n=min(len(x), x['sample_n'].iloc[0]),
          random_state=42
        ))
)
```

Step 4: Verifying Sample Size & Distribution

```
In [ ]: # Verify sample size = Original target sample size
print(f"Sample size: {len(df_sample_300k):,}")
```

```
In [ ]: # Verify distribution
df_sample_300k.groupby(strata_cols).size().describe()
```

Conclusion

This notebook delivers a reproducible stratified sample that preserves key distributional structure across product, time, and geography.

- The sampling design retains representational balance while reducing computational load.
- Stratum-level diagnostics indicate acceptable sparsity and stable allocation behavior.
- The exported 300k sample is ready for notebook 02 EDA and subsequent time-series pipelines.

```
In [ ]: df_sample_300k.to_parquet(  
        "data/processed/cfpb_sample_300k.parquet",  
        index=False  
    )
```